

Descrição do Projeto: Desenvolvimento de Aplicativo Web com angular e Consumo de APIs REST

Nome do aplicativo: AngularWeather

Título: App de Monitoramento Climático com angular

Objetivo:

Desenvolver um aplicativo web, utilizando o framework angular, que consuma dados de APIs REST externas. O objetivo é criar uma aplicação de monitoramento climático, onde o usuário pode visualizar as condições climáticas atuais e previsões de diferentes cidades, utilizando uma API de clima pública.

Versionamento Git

1. Crie um repositório público no GitHub, GitLab ou Bitbucket com o nome:
 - a. **AngularWeather**
2. O repositório deve conter:
 - a. Arquivo **LICENSE** (Creative Commons ou MIT)
 - b. Arquivo **README.md** com informações detalhadas sobre o projeto, contendo **ao menos 3 imagens de telas do projeto** (capriche na documentação)
 - c. Configure o arquivo **.gitignore** para garantir que o diretório **node_modules** não seja versionado
3. Submeta o código do projeto na branch **main**

Forma de Entrega:

- A resposta da atividade deverá ser apenas o link do repositório
- Não envie arquivos, apenas o link

Funcionalidades Principais:

1. **Autenticação de Usuário:**
 - o Implementação de um sistema de login e cadastro simples, utilizando serviços de autenticação com Firebase ou JSON Web Token (JWT), para que os usuários possam salvar suas cidades favoritas e acessar o histórico de pesquisas.
2. **Integração com API de Clima:**
 - o Consumo de uma API REST externa, como a OpenWeatherMap__ (<https://openweathermap.org/>), para buscar dados climáticos em tempo real. O aplicativo deverá exibir informações como temperatura, condições

atmosféricas (chuva, sol, nuvens), umidade e previsão dos próximos dias.

3. Interface de Usuário Responsiva:

- o Utilização dos componentes do angular para criar uma interface de usuário que seja simples, moderna e adaptada para dispositivos móveis.
- o Tela inicial com a exibição da cidade atual do usuário (baseada na localização do GPS ou uma cidade definida manualmente).

4. Busca de Cidades:

- o O usuário poderá buscar por qualquer cidade do mundo e visualizar as condições climáticas e a previsão. As cidades buscadas recentemente deverão ser armazenadas para rápido acesso posterior.

5. Favoritos:

- o Implementação de um sistema de favoritos, onde o usuário pode salvar suas cidades favoritas para monitoramento rápido. Essas informações serão salvas no armazenamento local ou em uma base de dados externa.

6. Exibição de Previsão por Dia e Hora:

- o Além das condições atuais, o aplicativo deverá exibir a previsão do tempo por hora (nas próximas 24 horas) e por dia (nos próximos 5 a 7 dias) para as cidades consultadas.

7. Localização Atual via GPS:

- o O aplicativo deverá oferecer a opção de localizar automaticamente a cidade do usuário via GPS e exibir as condições climáticas da localização atual.

8. Notificações Push:

- o Implementação de notificações push para alertar os usuários sobre mudanças climáticas significativas, como tempestades, ondas de calor ou alertas de frio extremo.

9. Cache e Performance:

- o Utilizar o armazenamento local para armazenar os dados climáticos recentes, de modo que o usuário possa visualizar as informações mesmo quando estiver offline ou com conexão instável.

Requisitos Técnicos:

- **Framework:** Angular

- **Linguagem:** TypeScript
- **Backend:** Firebase ou qualquer outro serviço de backend para autenticação e armazenamento de dados (opcional)
- **API de Clima:** OpenWeatherMap ou qualquer outra API pública de clima
- **Notificações:** Capacitor Push Notifications
- **Geolocalização:** Capacitor Geolocation para acessar a localização do usuário
- **Armazenamento Local:** Local Storage ou SQLite

Critérios de Avaliação:

1. **Funcionalidade:** O aplicativo deve ser capaz de consumir a API externa e exibir as informações de maneira correta e eficiente.
2. **Interface:** A interface do usuário deve ser intuitiva e responsiva.
3. **Usabilidade:** O aplicativo deve permitir uma navegação fácil e ser compreensível para o usuário.
4. **Integração de API:** As requisições REST devem ser implementadas corretamente, com tratamento de erros e respostas adequadas da API.
5. **Autenticação e Segurança:** Se a autenticação for implementada, ela deve ser segura e eficiente.
6. **Armazenamento Local e Offline:** O cache de dados locais deve funcionar corretamente para oferecer uma boa experiência offline.